ATIVIDADE - DESAFIO DE TESTES AUTOMATIZADOS (BDD, API & CONTRATO)

EcoSmart Waste Management (ESG)

Sistema Inteligente de Gestão de Resíduos e Reciclagem para Cidades Inteligentes

Aluno: Pedro Henrique De Souza Santiago **Stack: Java 17 | Spring Boot 3.2.5 | Cucumber 7.16 | RestAssured 5.4**

Repositório GitHub: https://github.com/PedroHSSantiago/ecosmart-waste-management

1. Integração ESG e Cobertura das 5 APIs do Projeto

A suíte automatizada valida todas as 5 APIs REST do ecossistema EcoSmart segundo os pilares Ambiental, Social e Governança:

Pilar ESG	API / Endpoint	Regras de Negócio e Comportamentos Validados nos Testes
Ambiental	`/api/pontos-coleta`	Lixeiras inteligentes com sensores IoT (ocupação 0-100%, bateria) e prevenção de transbordo.
Governança / Oper.	`/api/coletas-agendadas`	Rotas de caminhões com emissão de CO2, bloqueio de pontos inativos e motivo de cancelamento.
Social	`/api/usuarios-cidadaos`	Gamificação e recompensas para cidadãos por descarte reciclável (bronze, prata, ouro, platina).
Social / Governança	`/api/cooperativas-parceiras`	Gestão de cooperativas de catadores conveniadas, capacidade mensal e validação de CNPJ único.
Governança / Compl.	`/api/auditorias-ambientais`	Auditoria de metas ambientais exigindo plano_acao_corretiva obrigatório para status nao_conforme.

2. Matriz Completa dos 30 Cenários BDD em Gherkin (# language: pt)

Embora o requisito mínimo exija 3 cenários, foram implementados 30 cenários BDD (180 passos) cobrindo sucesso e falha:

#	Arquivo .feature	Cenário BDD (Dado / Quando / Então)	Tipo	Status	Contrato JSON Schem
01	`01_pontos_coleta`	Consultar ponto ativo com sensor IoT (PC-001)	Positivo	200 OK	`ponto-coleta-schema.json`
02	`01_pontos_coleta`	Filtrar pontos ativos com ocupação crítica > 80%	Positivo	200 OK	`pontos-coleta-lista-schema`
03	`01_pontos_coleta`	Cadastrar novo ponto de coleta inteligente (PC-011)	Positivo	201 Created	`ponto-coleta-schema.json`
04	`01_pontos_coleta`	Atualizar leitura de telemetria e ocupação IoT	Positivo	200 OK	`ponto-coleta-schema.json`
05	`01_pontos_coleta`	Remover ponto de coleta desativado (PC-010)	Positivo	204 NoCont	-
06	`01_pontos_coleta`	Falha ao cadastrar ponto com ocupação > 100%	Negativo	400 BadReq	`erro-api-schema.json`
07	`01_pontos_coleta`	Falha ao buscar ponto com código inexistente	Negativo	404 NotFnd	`erro-api-schema.json`
08	`02_coletas_agendadas`	Listar rotas de coleta com status agendado	Positivo	200 OK	`coletas-agendadas-lista`
09	`02_coletas_agendadas`	Agendar nova rota seletiva com estimativa de CO2	Positivo	201 Created	`coleta-agendada-schema`
10	`02_coletas_agendadas`	Concluir rota de coleta em andamento (ROTA-002)	Positivo	200 OK	`coleta-agendada-schema`
11	`02_coletas_agendadas`	Consultar detalhes de rota específica por ID	Positivo	200 OK	`coleta-agendada-schema`
12	`02_coletas_agendadas`	Remover rota de coleta cancelada (ROTA-004)	Positivo	204 NoCont	-
13	`02_coletas_agendadas`	Falha ao cancelar rota sem motivo obrigatório	Negativo	400 BadReq	`erro-api-schema.json`
14	`02_coletas_agendadas`	Falha ao agendar rota com ponto de coleta inativo	Negativo	400 BadReq	`erro-api-schema.json`
15	`03_usuarios_cidadaos`	Consultar cidadãos com pontuação > 500 pontos	Positivo	200 OK	`usuarios-cidadaos-lista`
16	`03_usuarios_cidadaos`	Registrar descarte reciclável e somar pontos	Positivo	200 OK	`usuario-cidadao-schema`
17	`03_usuarios_cidadaos`	Cadastrar novo cidadão no programa de reciclagem	Positivo	201 Created	`usuario-cidadao-schema`
18	`03_usuarios_cidadaos`	Consultar perfil completo de cidadão por e-mail	Positivo	200 OK	`usuario-cidadao-schema`
19	`03_usuarios_cidadaos`	Falha ao descartar em ponto em manutenção	Negativo	400 BadReq	`erro-api-schema.json`
20	`03_usuarios_cidadaos`	Falha ao buscar cidadão com e-mail inexistente	Negativo	404 NotFnd	`erro-api-schema.json`
21	`04_cooperativas`	Filtrar cooperativas de plástico (> 10.000 kg)	Positivo	200 OK	`cooperativas-lista-schema`
22	`04_cooperativas`	Cadastrar nova cooperativa parceira conveniada	Positivo	201 Created	`cooperativa-schema.json`
23	`04_cooperativas`	Consultar cooperativa específica por código	Positivo	200 OK	`cooperativa-schema.json`
24	`04_cooperativas`	Falha ao buscar cooperativa com código inexistente	Negativo	404 NotFnd	`erro-api-schema.json`
25	`04_cooperativas`	Falha ao cadastrar cooperativa com CNPJ duplicado	Negativo	409 Conflct	`erro-api-schema.json`
26	`05_auditoria_amb`	Consultar auditorias não conformes e plano de ação	Positivo	200 OK	`auditorias-lista-schema`
27	`05_auditoria_amb`	Registrar nova auditoria ambiental em conformidade	Positivo	201 Created	`auditoria-schema.json`
28	`05_auditoria_amb`	Consultar auditoria ambiental específica por código	Positivo	200 OK	`auditoria-schema.json`
29	`05_auditoria_amb`	Falha ao buscar auditoria com código inexistente	Negativo	404 NotFnd	`erro-api-schema.json`
30	`05_auditoria_amb`	Falha compliance: auditoria não conforme sem plano	Negativo	400 BadReq	`erro-api-schema.json`

3. Prints dos Cenários BDD em Gherkin e Contratos JSON Schema

src/test/resources/features/01_pontos_coleta.feature — Gherkin (PT)

```gherkin
# language: pt
Funcionalidade: Gestão de Pontos de Coleta Inteligentes (Pilar Ambiental ESG)
  Contexto: Dado que a base de dados do EcoSmart está inicializada com os registros padrões ESG

  Cenário: Consultar ponto de coleta ativo equipado com sensor IoT e validar contrato JSON Schema
    Quando eu envio uma requisição GET para "/api/pontos-coleta/PC-001"
    Então o status code da resposta deve ser 200
    E o corpo da resposta deve respeitar o contrato JSON Schema "schemas/ponto-coleta-schema.json"
    E o campo JSON "codigo_ponto" deve ser igual a "PC-001"
    E o campo JSON "sensor_iot.modelo" deve ser igual a "EcoSense X200"

  Cenário: Falha ao cadastrar ponto de coleta com nível de ocupação acima de 100 por cento
    Dado que eu possuo um payload inválido de ponto de coleta "PC-999" com nivel_ocupacao_percentual igual a 150
    Quando eu envio uma requisição POST para "/api/pontos-coleta" com o payload configurado
    Então o status code da resposta deve ser 400
    E o corpo da resposta deve respeitar o contrato JSON Schema "schemas/erro-api-schema.json"
    E o campo JSON "erro" deve ser igual a "Erro de Validação de Dados"
```

Print 1 - Especificação BDD em Gherkin cobrindo caminhos felizes (200/201) e tratamento de falhas (400/404) em 01_pontos_coleta.feature

src/test/resources/features/05_auditoria_ambiental.feature — Gherkin (PT)

```gherkin
# language: pt
Funcionalidade: Auditoria e Conformidade Ambiental (Pilar Governança ESG)
  Cenário: Registrar uma nova auditoria ambiental em conformidade com a meta municipal
    Dado que eu possuo os dados de uma nova auditoria "AUD-005" com valor medido 1500.0 para a meta 1200.0
    Quando eu envio uma requisição POST para "/api/auditorias-ambientais" com o payload configurado
    Então o status code da resposta deve ser 201
    E o corpo da resposta deve respeitar o contrato JSON Schema "schemas/auditoria-ambiental-schema.json"
    E o campo decimal JSON "percentual_atingimento" deve ser igual a 125.0

  Cenário: Falha de compliance ao registrar auditoria não conforme sem plano de ação corretiva
    Dado que eu possuo uma auditoria não conforme "AUD-006" abaixo da meta sem plano de ação corretiva
    Quando eu envio uma requisição POST para "/api/auditorias-ambientais" com o payload configurado
    Então o status code da resposta deve ser 400
    E o corpo da resposta deve respeitar o contrato JSON Schema "schemas/erro-api-schema.json"
    E o campo JSON "mensagem" deve conter "exigem obrigatoriamente um plano_acao_corretiva"
```

Print 2 - Cenários BDD de Governança e Compliance Ambiental em 05_auditoria_ambiental.feature

src/test/resources/schemas/ponto-coleta-schema.json & EcoSmartApiSteps.java — JSON Schema + RestAssured

```
// 1. Contrato JSON Schema (src/test/resources/schemas/ponto-coleta-schema.json):
{ "$schema": "http://json-schema.org/draft-07/schema#", "type": "object",
  "required": ["codigo_ponto", "tipo_residuo", "localizacao", "bairro",
               "capacidade_litros", "nivel_ocupacao_percentual", "status"],
  "properties": {
    "nivel_ocupacao_percentual": { "type": "integer", "minimum": 0, "maximum": 100 },
    "status": { "type": "string", "enum": ["ativo", "manutencao", "inativo"] }
  }, "additionalProperties": false }

// 2. Validação Automatizada no Step Definition (EcoSmartApiSteps.java):
@E("o corpo da resposta deve respeitar o contrato JSON Schema {string}")
public void validarContratoJsonSchema(String schemaPath) {
    response.then().assertThat().body(JsonSchemaValidator.matchesJsonSchemaInClasspath(schemaPath));
}
```

Print 3 - Configuração dos Contratos JSON Schema (11 schemas) e validação com RestAssured JsonSchemaValidator

4. Evidências da Execução Bem-Sucedida dos Testes (Maven & Cucumber Report)

PowerShell Terminal - mvn clean test (Execução Real da Suíte BDD & API) Exit Code: 0

```
[INFO] Scanning for projects...
[INFO] ------------< br.com.ecosmart:ecosmart-waste-management >--------------
[INFO] Building EcoSmart Waste Management 1.0.0
[INFO] --- surefire:3.2.5:test (default-test) @ ecosmart-waste-management ---
[INFO] Running br.com.ecosmart.CucumberTestRunner
 :: Spring Boot :: (v3.2.5) - Tomcat started on port 65352 (http) with context path ''
Started CucumberSpringConfiguration in 2.417 seconds (process running for 3.917)

Cenário: Consultar ponto de coleta ativo equipado com sensor IoT e validar contrato JSON Schema
  [OK] Dado que a base de dados do EcoSmart está inicializada com os registros padrões ESG
  [OK] Quando eu envio uma requisição GET para "/api/pontos-coleta/PC-001"
  [OK] Então o status code da resposta deve ser 200
  [OK] E o corpo da resposta deve respeitar o contrato JSON Schema "schemas/ponto-coleta-schema.json"
...
Cenário: Falha de compliance ao registrar auditoria não conforme sem plano de ação corretiva
  [OK] Quando eu envio uma requisição POST para "/api/auditorias-ambientais" com o payload configurado
  [OK] Então o status code da resposta deve ser 400
  [OK] E o corpo da resposta deve respeitar o contrato JSON Schema "schemas/erro-api-schema.json"

30 Scenarios (30 passed)
180 Steps (180 passed)
0m7,029s
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.353 s -- in br.com.ecosmart.CucumberTestRunner
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  10.503 s | Finished at: 2026-11-03T22:47:26-03:00
```

Print 4 - Evidência de execução no terminal com 100% de aprovação nos 30 cenários BDD e 180 passos (BUILD SUCCESS)

Cucumber HTML Report - target/cucumber-reports/cucumber.html STATUS: 100% PASSED (30/30)

100%	30 / 30	180 / 180	11 / 11
TAXA DE APROVAÇÃO	CENÁRIOS BDD PASSANDO	PASSOS GHERKIN EXECUTADOS	SCHEMAS JSON VALIDADOS

[OK] 01_pontos_coleta.feature - Gestão de Pontos de Coleta Inteligentes (Pilar Ambiental ESG)	7 / 7 Cenários (PASSED)
[OK] 02_coletas_agendadas.feature - Gestão de Coletas Agendadas e Rotas (Pilar Governança ESG)	7 / 7 Cenários (PASSED)
[OK] 03_usuarios_cidadaos.feature - Engajamento de Cidadãos e Recompensas (Pilar Social ESG)	6 / 6 Cenários (PASSED)
[OK] 04_cooperativas_parceiras.feature - Cooperativas Parceiras (Pilar Social/Governança ESG)	5 / 5 Cenários (PASSED)
[OK] 05_auditoria_ambiental.feature - Auditoria e Conformidade Ambiental (Pilar Governança ESG)	5 / 5 Cenários (PASSED)

Print 5 - Painel do Relatório HTML do Cucumber gerado em target/cucumber-reports/cucumber.html

.github/workflows/ci-tests.yml & Dockerfile CI/CD & Docker

```
# Pipeline GitHub Actions (.github/workflows/ci-tests.yml) + Dockerfile Multi-Stage:
- name: Executar Testes Automatizados BDD e Contrato de API
  run: mvn clean test -B
- name: Publicar Relatório de Evidências Cucumber (HTML & JSON)
  uses: actions/upload-artifact@v4  # path: target/cucumber-reports/
```

Print 6 - Automação de execução contínua via GitHub Actions CI/CD e containerização Docker